

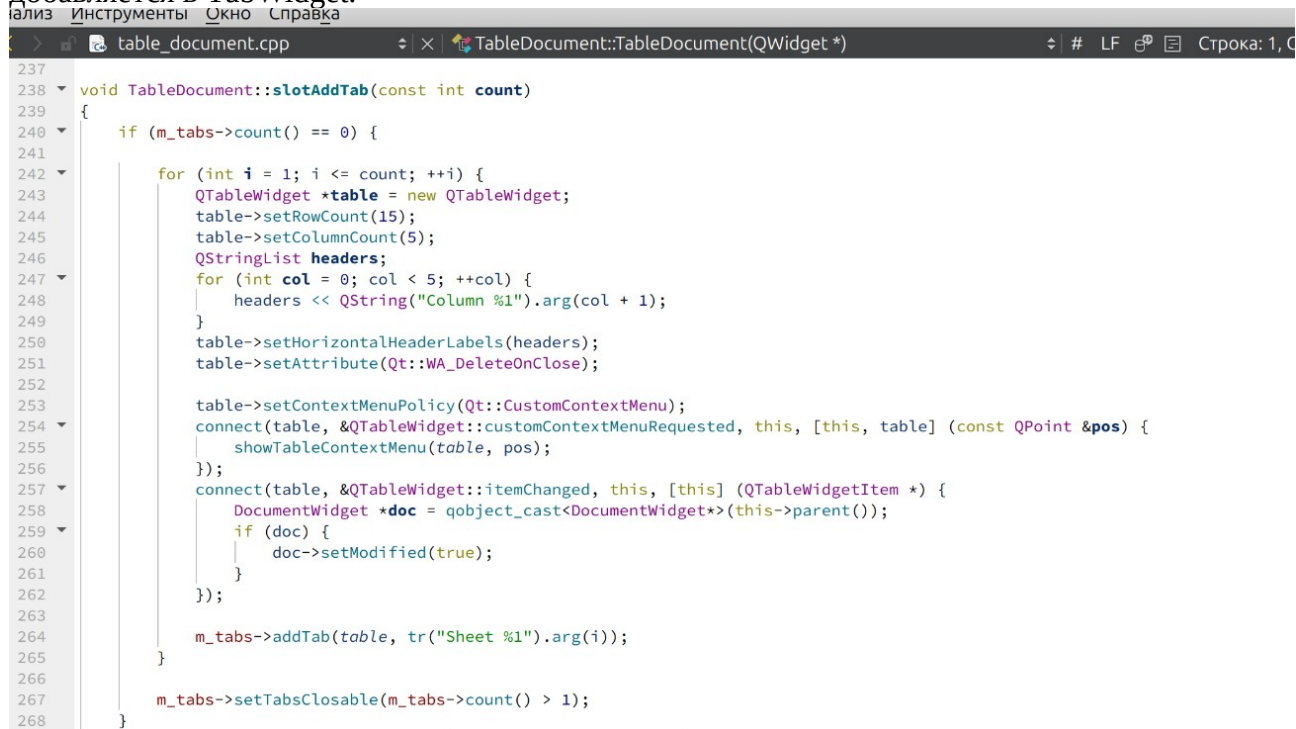
Практическая работа №5

Выполнила: Ушакова Е.А. гр. ЗП-31

Все исходные файлы лабораторных работ лежат в папке репозитория по ссылке

<https://github.com/irbi42/uni/tree/main/vp/labs>

1) Добавление таблицы. Создается новый виджет таблицы, для него задаются заголовки и подключается слот кастомного меню и индикации изменения документа. Затем новая таблица добавляется в TabWidget.



```
237
238 void TableDocument::slotAddTab(const int count)
239 {
240     if (m_tabs->count() == 0) {
241
242         for (int i = 1; i <= count; ++i) {
243             QTableWidget *table = new QTableWidget;
244             table->setRowCount(15);
245             table->setColumnCount(5);
246             QStringList headers;
247             for (int col = 0; col < 5; ++col) {
248                 headers << QString("Column %1").arg(col + 1);
249             }
250             table->setHorizontalHeaderLabels(headers);
251             table->setAttribute(Qt::WA_DeleteOnClose);
252
253             table->setContextMenuPolicy(Qt::CustomContextMenu);
254             connect(table, &QTableWidget::customContextMenuRequested, this, [this, table] (const QPoint &pos) {
255                 showTableContextMenu(table, pos);
256             });
257             connect(table, &QTableWidget::itemChanged, this, [this] (QTableWidgetItem *) {
258                 DocumentWidget *doc = qobject_cast<DocumentWidget*>(this->parent());
259                 if (doc) {
260                     doc->setModified(true);
261                 }
262             });
263
264             m_tabs->addTab(table, tr("Sheet %1").arg(i));
265         }
266
267         m_tabs->setTabsClosable(m_tabs->count() > 1);
268     }
```

2) Слот реализации контекстного меню. Создаются объекты QAction для соответствующих пунктов меню. Для каждого из действий прописывается действия добавления или удаления строк и столбцов соответственно. Каждое действие испускает сигнал об изменении содержимого.

```

53 void TableDocument::showTableContextMenu(QTableWidget *table, const QPoint &pos) {
54     QMenu menu;
55     QAction *actionAddRow = menu.addAction(tr("Add Row"));
56     QAction *actionRemoveRow = menu.addAction(tr("Remove Row"));
57     QAction *actionAddColumn = menu.addAction(tr("Add Column"));
58     QAction *actionRemoveColumn = menu.addAction(tr("Remove Column"));
59     QAction *actionCreateBarChart = menu.addAction(tr("Create Bar Chart"));
60
61
62     QAction *selectedAction = menu.exec(table->mapToGlobal(pos));
63     if (!selectedAction) return;
64     if (selectedAction == actionAddRow) {
65         table->setRowCount(table->rowCount() + 1);
66         emit contentChanged(true);
67     } else if (selectedAction == actionRemoveRow && table->rowCount() > 0) {
68         table->setRowCount(table->rowCount() - 1);
69         emit contentChanged(true);
70     } else if (selectedAction == actionAddColumn) {
71         int newColIndex = table->columnCount();
72         table->setColumnCount(newColIndex + 1);
73         table->setHorizontalHeaderItem(newColIndex, new QTableWidgetItem(QString("Column %1").arg(newColIndex + 1)));
74         emit contentChanged(true);
75     } else if (selectedAction == actionRemoveColumn && table->columnCount() > 0) {
76         int colCount = table->columnCount();
77         table->setColumnCount(colCount - 1);
78         emit contentChanged(true);
79     } else if (selectedAction == actionCreateBarChart && !table->selectedItems().isEmpty()) {
80         auto *chart = new ChartBuilder(table);
81         chart->buildBarChart(table->selectionModel()->selectedIndexes());
82         QGraphicsView *chartView = new QGraphicsView(chart);
83         int tabIndex = m_tabs->addTab(chartView, tr("Diagramm"));
84         m_tabs->setCurrentIndex(tabIndex);
85         emit contentChanged(true);
86     }
87 }
88
89

```

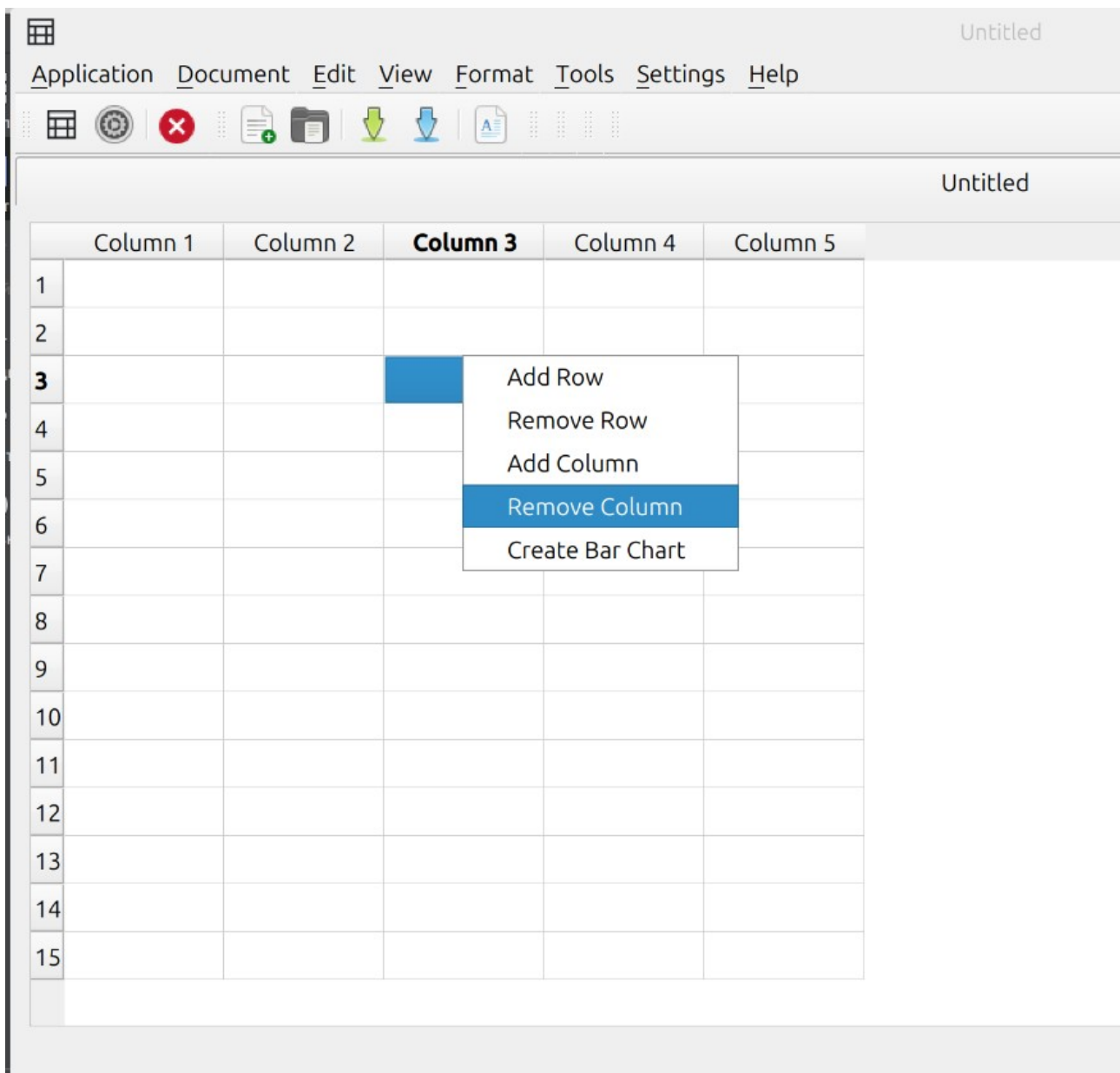
В конструкторе документа этот сигнал связывается с функцией

```

36
37 DocumentWidget::DocumentWidget(QWidget *parent)
38     : TableDocument(parent)
39     , m_modified{false}
40     , m_url{QUrl()}
41 {
42     setAttribute(Qt::WA_DeleteOnClose);
43     connect(this, &TableDocument::contentChanged, this, &DocumentWidget::setModified);
44 }
45

```

Результат работы:



Ответы на контрольные вопросы

Что такое MDI и как он реализован в Qt?

MDI (Multiple Document Interface) — интерфейс для работы с несколькими дочерними окнами внутри одного главного. В Qt реализуется через классы `QMdiArea` (рабочая область) и `QMdiSubWindow` (дочерние окна). Дочерние окна можно каскадировать, располагать плиткой, сворачивать.

Как реализуется контекстное меню в Qt?

Создаётся объект `QMenu`, добавляются `QAction`. Вызов меню — через `exec(QCursor::pos())` или переопределение метода `contextMenuEvent()`. Также можно установить `setContextMenuPolicy(Qt::CustomContextMenu)` и подключиться к сигналу `customContextMenuRequested()`.

Какие сигналы и слоты используются для отслеживания изменений в таблице?

Сигналы `QAbstractItemModel`: `dataChanged()`, `rowsInserted()`, `rowsRemoved()`, `modelReset()`. У `QTableWidget`: `itemChanged(QTableWidgetItem*)`, `cellChanged(row, col)`.

Как работает механизм `modifiedChanged` и зачем он нужен?

Это флаг (обычно `isModified`), который становится `true` при любом изменении документа и `false` после сохранения. Нужен для отслеживания несохранённых изменений, чтобы при закрытии окна предложить сохранить файл.

Как реализуется связь между `TableDocument` и `DocumentWidget`?

`DocumentWidget` наследуется от `TableDocument`. Связь реализована через:

В `TableDocument` при изменении таблицы (`itemChanged`) вызывается лямбда, которая через `qobject_cast<DocumentWidget*>(this->parent())` получает родителя и вызывает `doc->setModified(true)`

Сигнал `contentChanged` из `TableDocument` подключён к слоту `setModified` в `DocumentWidget`. `DocumentWidget` управляет флагом `modified` и `url`, а `TableDocument` управляет вкладками и таблицами.